

SMF ツール (SmfPlayer / MidiRecorder / TempoMapExtractor)

このページでは、Standard MIDI File (SMF) の再生・記録・テンポ解析を行うコンポーネントとユーティリティについて説明します。

名前空間:

- コンポーネント: `jp.kshoji.unity.midi`
- ユーティリティ: `jp.kshoji.unity.midi.util`

配置:

- `Assets/MIDI/Scripts/Gameplay/SmfPlayer.cs`
- `Assets/MIDI/Scripts/Gameplay/MidiRecorder.cs`
- `Assets/MIDI/Scripts/MidiSequenceAsset.cs`
- `Assets/MIDI/Scripts/UmpSequenceAsset.cs`
- `Assets/MIDI/Scripts/Editor/UmpImportUtility.cs` / `UmpAssetImportMenu.cs`
- `Assets/MIDI/Scripts/Utilities/Smf/TempoMapExtractor.cs` 等

概要

コンポーネント / ユーティリティ	役割
MidiSequenceAsset	SMF バイナリを ScriptableObject として保持
UmpSequenceAsset	.midi2 (UMP MIDI Clip) バイナリを ScriptableObject として保持
SmfPlayer	SMF / Sequence を SequencerImpl で再生し MidiManager へ出力
MidiRecorder	ライブ MIDI 入力を SMF として記録
TempoMapExtractor	Sequence からテンポ・拍子記号を時系列抽出

いずれも既存の `jp.kshoji.midisystem` (`Sequence` / `SequencerImpl`) をラップします。低レイヤーの SMF 入出力 API の詳細は [SMF / シーケンシング](#) を参照してください。

MidiSequenceAsset

SMF データを Unity アセットとして保持する ScriptableObject です。`SmfPlayer` の再生ソースや `MidiRecorder` の保存先として使います。

作成方法

- Inspector: `Assets > Create > MIDI > Sequence Asset`
- コード: `MidiSequenceAsset.CreateFromSequence(sequence, sourcePath)`

API

メソッド	説明
<code>SetSmfData(byte[] data, string sourcePath)</code>	SMF バイナリを設定
<code>ToSequence()</code>	<code>Sequence</code> に変換
<code>CreateFromSequence(Sequence, string)</code>	<code>Sequence</code> からアセットを生成

UmpSequenceAsset

`.midi2` (UMP MIDI Clip / SMF2CLIP) バイナリを Unity アセットとして保持する ScriptableObject です。
`UmpSequencer` や Scriptable Audio の `MidiDspUmpSequenceScheduler` の再生ソースとして使います。

作成方法

- Inspector: `Assets > Create > MIDI > UMP Sequence Asset` (空アセット)
- ファイルから: `Assets > Create > MIDI > Import UMP Sequence Asset From File` (`.midi2` を選択)
- コード: `UmpSequenceAsset.CreateFromSequence(umpSequence, sourcePath) / UmpImportUtility`

API

メソッド / プロパティ	説明
<code>SetMidi2Data(byte[] data, string sourcePath, int index)</code>	<code>.midi2</code> バイナリを設定
<code>ToUmpSequence()</code>	設定クリップを <code>UmpSequence</code> に変換
<code>ToUmpSequenceList()</code>	ファイル内の全クリップを変換
<code>CreateFromSequence(UmpSequence, string)</code>	<code>UmpSequence</code> からアセットを生成
<code>ClipIndex</code>	マルチクリップファイル内のインデックス (通常 0)

Scriptable Audio での利用例

```
var asset = /* Inspector で割り当てた UmpSequenceAsset */;  
var bootstrap = GetComponent<MidiDspUmpSequenceBootstrap>();  
bootstrap.sequence = asset.ToUmpSequence();  
bootstrap.EnsureReady();  
bootstrap.Scheduler.Play();
```

DSP 同期再生の詳細は [Unity エコシステム統合 — UMP シーケンス](#) を参照してください。ウォールクロック再生は [MIDI 2.0 — UmpSequencer](#) を参照してください。

SmfPlayer

`SequencerImpl` を Unity ライフサイクルに統合し、SMF をゲーム時間に同期して再生するコンポーネントです。出力は内部の `MidiManagerReceiverBridge` 経由で `MidiManager` に送られます。

セットアップ

- 1. `GameObject` に `SmfPlayer` をアタッチ
- 2. `Sequence Asset` に `MidiSequenceAsset` を割り当て（または実行時に `SetSequence()`）
- 3. `Output Device Id` を指定（空 = 最初の出力デバイス）
- 4. シーン bootstrap で `MidiManager.InitializeMidi()` を呼び出す

MPTK 音声出力 (`FEATURE_USE_MPTK`)

ハードウェア MIDI デバイスの代わりに MPTK シンセへ出力する場合:

- 1. Project Settings で `FEATURE_USE_MPTK` を有効化
- 2. 同じ `GameObject` に `MptkSmfPlayerOutput` を追加
- 3. Play モードで `Configure()` が `outputDeviceId` を `mptk:internal` に設定

詳細は [Maestro / MPTK 統合 — SmfPlayer 連携](#) を参照してください。

エディタ SMF 試聴

`Window > MIDI > SMF Preview` で `.mid` を読み込んだ後、**Preview Audio (MPTK)** から Play Mode 試聴できます（`FEATURE_USE_MPTK` 必須）。詳細は [Maestro / MPTK 統合 — エディタ SMF 試聴](#) を参照してください。

主な設定

フィールド	説明
<code>sequenceAsset</code>	再生する SMF アセット
<code>group</code>	MIDI グループ (0-15)
<code>outputDeviceId</code>	出力デバイス ID（空 = 自動選択）
<code>outputChannel</code>	全メッセージをこのチャンネルへマップ（-1 = 元のチャンネルを維持）
<code>playOnAwake</code>	Awake 後に自動再生
<code>loop / loopCount</code>	ループ再生（-1 = 無限）
<code>tempoBpm</code>	再生テンポ（実行中の変更も反映）

操作 API

メソッド	説明
<code>Play()</code>	再生開始（一時停止中は位置を維持して再開）
<code>Pause()</code>	一時停止
<code>Stop()</code>	停止して先頭へ戻す
<code>Seek(float timeSeconds)</code>	再生位置をシーク
<code>SetTrackMute(index, mute)</code>	トラック Mute
<code>SetTrackSolo(index, solo)</code>	トラック Solo
<code>SetSequenceAsset(asset)</code>	アセットを差し替えて再読み込み
<code>SetSequence(sequence)</code>	実行時 Sequence を直接指定

状態プロパティ

プロパティ	説明
<code>State</code>	<code>Stopped</code> / <code>Playing</code> / <code>Paused</code>
<code>CurrentTimeSeconds</code>	現在の再生位置（秒）
<code>TotalDurationSeconds</code>	総再生時間（秒、 <code>TempoMapExtractor</code> ベース）

イベント

UnityEvent	タイミング
<code>onPlaybackStarted</code>	再生開始
<code>onPlaybackPaused</code>	一時停止
<code>onPlaybackStopped</code>	停止
<code>onPlaybackFinished</code>	ループなしで末尾到達

使用例

```
using jp.kshoji.unity.midi;
using UnityEngine;

public sealed class SmfPlaybackController : MonoBehaviour
{
    public SmfPlayer player;

    void Start()
    {
        MidiManager.Instance.InitializeMidi(() => player.Play());
    }

    public void OnSeek(float normalized)
    {
        player.Seek(player.TotalDurationSeconds * normalized);
    }
}
```

MidiRecorder

ライブ MIDI 入力（Note / CC / Program Change / Pitch Bend / Aftertouch / SysEx / システムリアルタイム）を `Sequence` に記録し、`.mid` ファイルまたは `MidiSequenceAsset` として保存します。

現行実装では `Time.time` ベースの自前タイムスタンプ方式を採用しています（`SequencerImpl` 録音 API 統合は将来拡張）。

セットアップ

- 1. `GameObject` に `MidiRecorder` をアタッチ
- 2. `registerWithMidiManager = true`（デフォルト）で `MidiManager` に登録
- 3. 必要に応じて `deviceIdFilter` / `channelFilter` を設定
- 4. `StartRecording()` / `StopRecording()` で記録制御

フィルタチェーン経由で使う場合は `registerWithMidiManager = false` にし、上流フィルタの `forwardTargets` へ配置してください（[ゲームプレイ向けコンポーネント](#)と同様）。

主な設定

フィールド	説明
<code>tempoBpm</code>	記録時のテンポ（Tick 計算に使用）
<code>ticksPerQuarter</code>	分解能（PPQ、デフォルト 480）
<code>deviceIdFilter</code>	記録対象デバイス（空 = 全デバイス）
<code>channelFilter</code>	記録対象チャンネル（空 = 全チャンネル）

API

メソッド	説明
<code>StartRecording()</code>	記録開始
<code>StopRecording()</code>	記録停止
<code>ClearRecording()</code>	記録内容をクリア
<code>GetSequence()</code>	記録済み <code>Sequence</code> を取得
<code>SaveToFile(path)</code>	<code>.mid</code> ファイルへ保存
<code>SaveToAsset(assetPath)</code>	<code>MidiSequenceAsset</code> を作成（Editor のみ）

記録フォーマット

- SMF Format 1 (テンポトラック + データトラック)
- 記録対象: Note On/Off、Control Change、Program Change、Pitch Bend、Channel/Poly Aftertouch、SysEx / System Common、MIDI リアルタイム (Clock / Start / Stop 等)
- Raw UMP は SMF に書き出さない (`onMessage` で別途処理)

使用例

```
using jp.kshoji.unity.midi;
using UnityEngine;

public sealed class RecordingExample : MonoBehaviour
{
    public MidiRecorder recorder;
    public SmfPlayer player;
    public MidiSequenceAsset recordedAsset;

    public void RecordAndPlay()
    {
        recorder.StartRecording();
        // ... MIDI 演奏 ...
        recorder.StopRecording();

#if UNITY_EDITOR
        recordedAsset = recorder.SaveToAsset("Assets/Recorded.mid.asset");
        player.SetSequenceAsset(recordedAsset);
        player.Play();
#endif
    }
}
```

TempoMapExtractor

Sequence または SMF データからテンポ変更・拍子記号を時系列で抽出する静的ユーティリティです。
SmfPlayer.TotalDurationSeconds の計算や、譜面生成の基盤として使えます。

API

```
using jp.kshoji.unity.midi.util;  
using jp.kshoji.midisystem;  
  
TempoMap map = TempoMapExtractor.Extract(sequence);  
// または  
TempoMap map = TempoMapExtractor.Extract(smfBytes);  
TempoMap map = TempoMapExtractor.Extract("/path/to/file.mid");
```

TempoMap プロパティ / メソッド

名前	説明
TempoChanges	テンポ変更リスト
TimeSignatures	拍子記号リスト
TotalDurationSeconds	総再生時間（秒）
TotalTicks	総 Tick 数
TicksPerQuarter	分解能
GetBpmAt(tick)	指定 Tick 時点の BPM
TickToSeconds(tick)	Tick → 秒
SecondsToTick(seconds)	秒 → Tick

テンポメタイイベントが存在しない SMF では、120 BPM（500000 μ s/qn）をデフォルトとして扱います。

使用例

```
var map = TempoMapExtractor.Extract(sequence);
Debug.Log($"Duration: {map.TotalDurationSeconds:F2}s, BPM at 0: {map.GetBpmAt(0)}");

foreach (var signature in map.TimeSignatures)
{
    Debug.Log($"{{signature.TimeSeconds:F2}}s -> {{signature.Numerator}}/{{signature.Denom
```

MeasureTimeUtility

TempoMap 上の小節 / 拍 Tick 計算と、小節 ↔ 時間の変換。

場所: Assets/MIDI/Scripts/Utilities/Smf/MeasureTimeUtility.cs

メソッド	説明
TicksPerBar(ppqn, numerator, denominator)	1 小節の Tick 長
TicksForBars(...)	N 小節の長さ
TicksPerBeat(...)	単純拍子における拍長（小節 / 分子）
MeasureToStartSeconds / MeasureToEndSeconds	1 始まりの小節番号 → 秒
TryResolveMeasureRange / TryGetMeasureRangeMs	両端を含む小節範囲
TimeSecondsToMeasure	秒 → 1 始まりの小節番号

小節 Tick の式: $ppqn * 4 * numerator / denominator$ （整数除算）。

TupletUtility / SwingUtility

場所: Assets/MIDI/Scripts/Utilities/Smf/TupletUtility.cs , SwingUtility.cs

TupletUtility

固定 PPQN グリッド上の有理連符（例: 3:2 の 8 分音符三連符）。ステップ Tick は整数除算を使うため、範囲内の index→tick→index は往復可能です。

メソッド	説明
UnitTicks / GroupDurationTicks / StepTicks	単位・グループ幅・ステップごとの Tick
TickAtIndex / IndexAtTick	双方向マッピング
QuantizeToGroup	Tick を最も近い連符ステップにスナップ
IsIndexTickReversible	全インデックスで往復可能性を検証

SwingUtility

再生時スイング: 編集 Tick を書き換えずに偶数オフビートを遅らせます。swingAmount ≈ 0.33 で三連符風のフィーリングになります。

メソッド	説明
UnitTicksFromDivision(ppqn, division)	単位長（4 = 4 分、8 = 8 分、...）
ApplySwing(editTick, unitTicks, amount)	編集 Tick → スイング後の再生 Tick
ApplySwing(..., ppqn, swingDivision, amount)	便利なオーバーロード

MidiMetaMessageFactory

よく使う SMF `MetaMessage` ペイロードの生成と読み取り。

場所: `Assets/MIDI/Scripts/Utilities/Smf/MidiMetaMessageFactory.cs`

メソッド	説明
<code>CreateTempo(bpm)</code>	Set Tempo メタ (四部音符あたりの μ s)
<code>CreateTimeSignature(numerator, denominator, ...)</code>	Time Signature メタ
<code>CreateTrackName(name)</code>	Track / Sequence Name (UTF-8)
<code>DenominatorToExponent</code> / <code>ExponentToDenominator</code>	SMF の 2 の冪変換
<code>TryReadTempoBpm(meta, out bpm)</code>	Tempo メタから BPM を読む

ソースファイル

ファイル	役割
MidiSequenceAsset.cs	SMF ScriptableObject
UmpSequenceAsset.cs	UMP .midi2 ScriptableObject
Editor/UmpImportUtility.cs / UmpAssetImportMenu.cs	.midi2 インポート
SmfPlayer.cs / SmfPlayerState.cs	SMF 再生コンポーネント
MidiManagerReceiverBridge.cs	Sequencer → MidiManager ブリッジ
MidiRecorder.cs	録音コンポーネント
MidiRecordingSession.cs	録音セッション (Tick 計算・SMF 書き出し)
TempoMapExtractor.cs	テンポマップ抽出
TempoMap.cs / TempoMapEntry.cs / TimeSignatureEntry.cs	データモデル
MeasureTimeUtility.cs	小節 / 小節 Tick ヘルパー
TupletUtility.cs / SwingUtility.cs	連符とスイングタイミング
MidiMetaMessageFactory.cs	SMF メタ生成

関連ドキュメント

- [SMF / シーケンシング \(jp.kshoji.midisystem\)](#)
- [MIDI 2.0 / UMP](#)
- [Unity エコシステム統合 — UMP シーケンス](#)
- [ゲームプレイ向けコンポーネント](#)
- [MIDI 1.0 ランタイム \(MidiManager\)](#)
- [サンプル](#)